

Name(s) _____ Period _____ Date _____

Asphalt Art Project Planning Guide



Cities use asphalt art to improve public safety, inspire their residents and visitors, and brighten communities. Your goal is to create asphalt art to revitalize The Neighborhood and bring the community together with the help of the Painter.

Project Requirements

Use your knowledge of object-oriented programming, algorithms, the problem solving process, and decomposition strategies to create asphalt art:

- **Create a new subclass** – Create at least one new subclass of the PainterPlus class that is used for a component of the asphalt art design.
- **Plan an algorithm** – Use the problem solving process and decomposition strategies to plan an algorithm that incorporates a combination of sequencing, selection, and/or iteration.
- **Write a method** – Write at least one method in a PainterPlus subclass that contributes to a component of the asphalt art design.
- **Document your code** – Use comments to explain the purpose of the methods and code segments.

Project Rubric

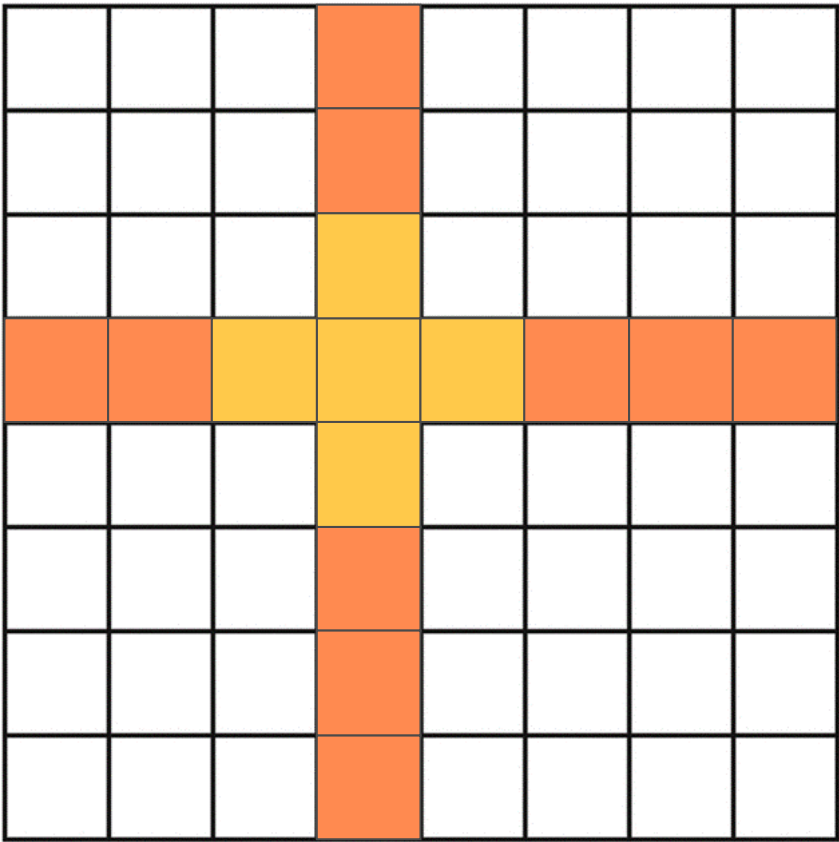
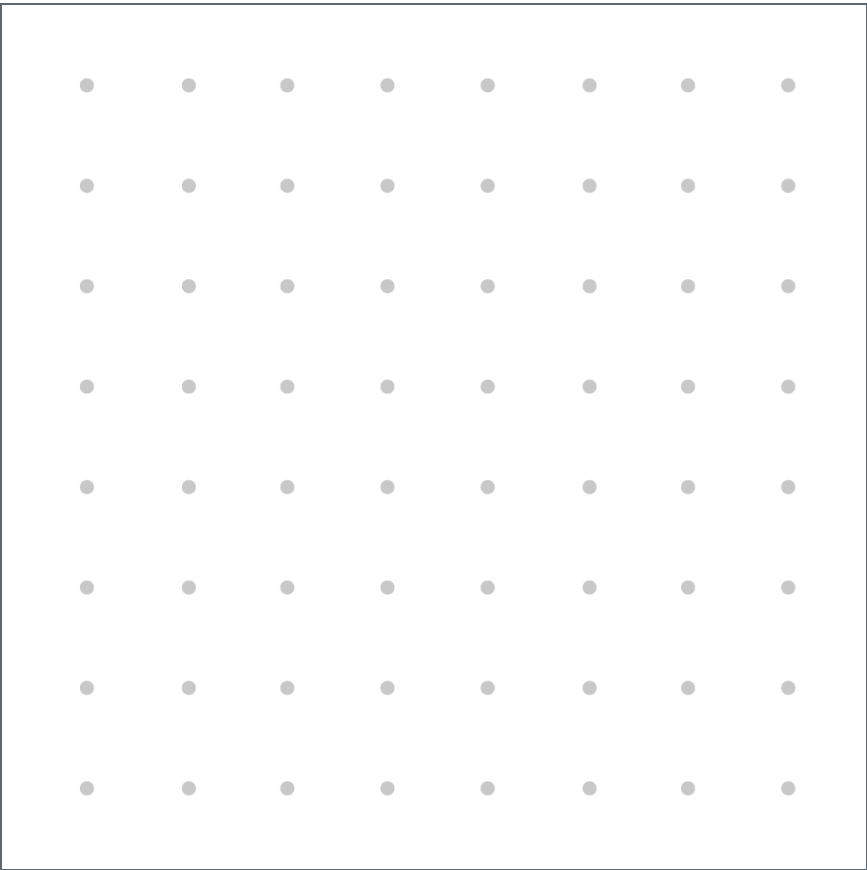
Category	Extensive Evidence	Convincing Evidence	Limited Evidence	No Evidence
Object-Oriented Programming	Creates a new subclass of the PainterPlus class that is used as a component of the asphalt art design AND Creates an additional subclass of the PainterPlus class that is used as a component of the asphalt art design OR Creates an additional subclass that extends the PainterPlus subclass that is used as a component of the asphalt art design	Creates a new subclass of the PainterPlus class that is used for a component of the asphalt art design	Creates a new subclass of the PainterPlus class but does not use it for a component of the asphalt art design	Does not create a new subclass of the PainterPlus class
Algorithm Development	Uses the problem solving process and decomposition strategies to plan more than one algorithm that incorporates a combination of sequencing, selection, and/or iteration	Uses the problem solving process and decomposition strategies to plan an algorithm that incorporates a combination of sequencing, selection, and/or iteration	Uses the problem solving process and decomposition strategies to plan an algorithm that only incorporates sequencing	Does not use the problem solving process or decomposition strategies to plan an algorithm

Category	Extensive Evidence	Convincing Evidence	Limited Evidence	No Evidence
Methods	Writes a method in a PainterPlus subclass that contributes to a component of the asphalt art design AND Writes a method in the additional subclass of the PainterPlus class that contributes to a component of the asphalt art design OR Writes a method in the additional subclass that extends the PainterPlus subclass that contributes to a component of the asphalt art design	Writes a method in a PainterPlus subclass that contributes a component of the asphalt art design	Writes a method in a PainterPlus class that contributes to a component of the asphalt art design	Does not write a new method in the PainterPlus class or any subclasses of the PainterPlus class
Documentation	Uses single and multi-line comments to explain the functionality and purpose of methods and code segments in the program	Uses single-line comments to explain the functionality and purpose of methods and code segments in the program	Uses single-line comments to explain the functionality and purpose of methods in the program	Does not use comments to document the functionality and purpose of methods or code segments in the program

Step 1: Sketch Your Design

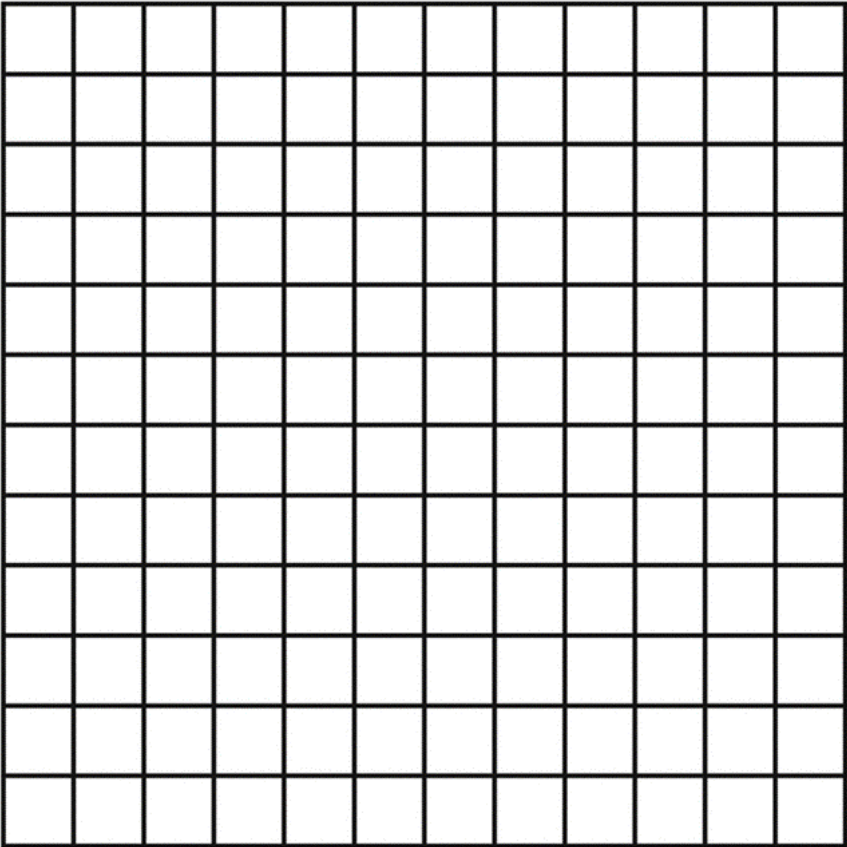
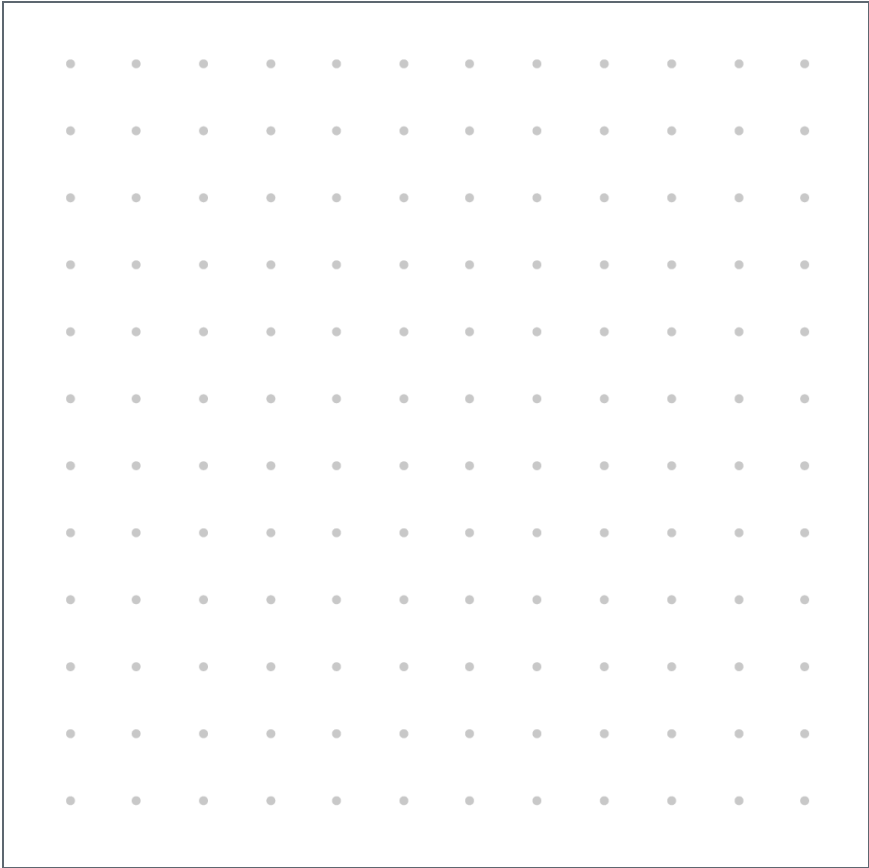
Choose one of the available grid sizes for your design.

8 x 8 Grid

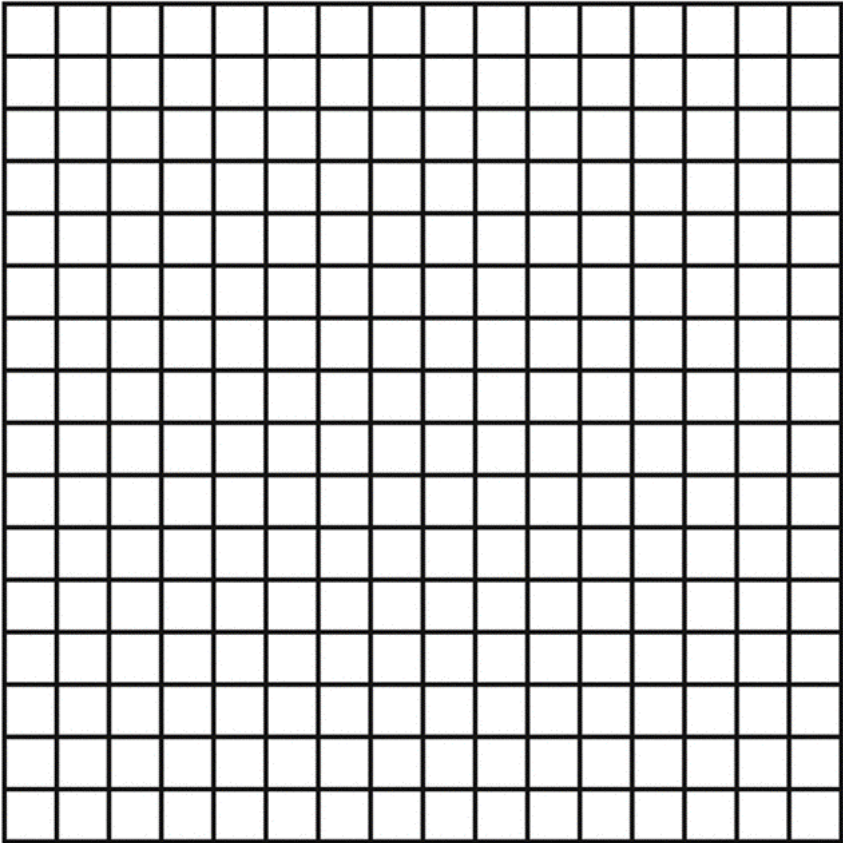
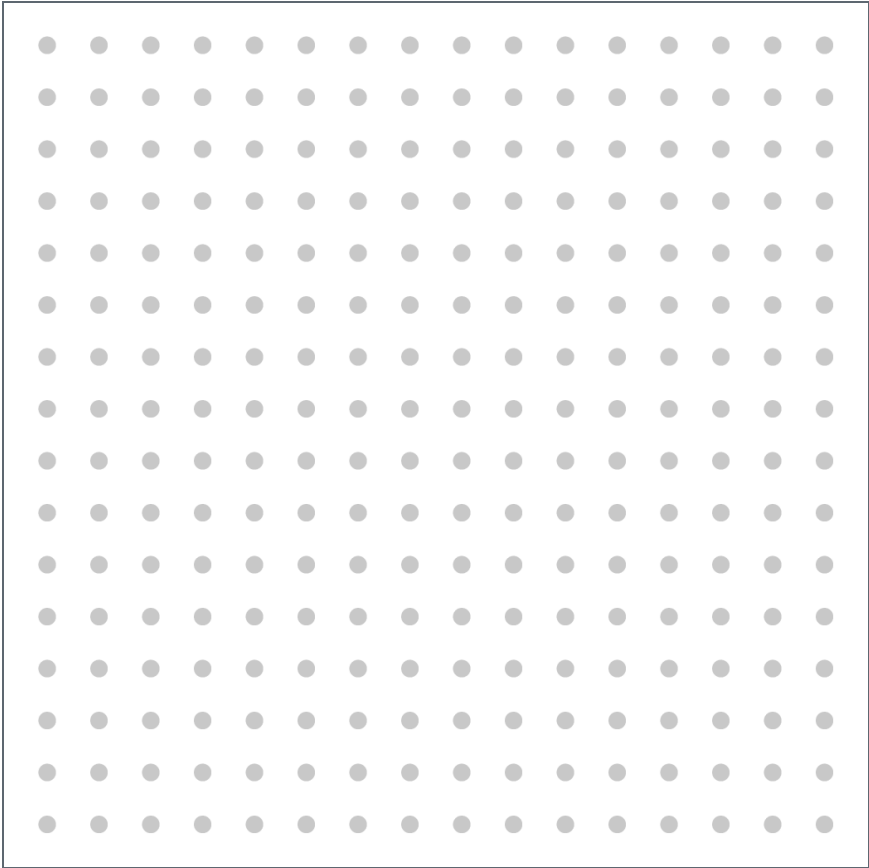


Gold = sun core (`SunPainter.paintSunCore`)
Orange = rays (`SunPainter.paintRay`, called 4 times)

12 x 12 Grid

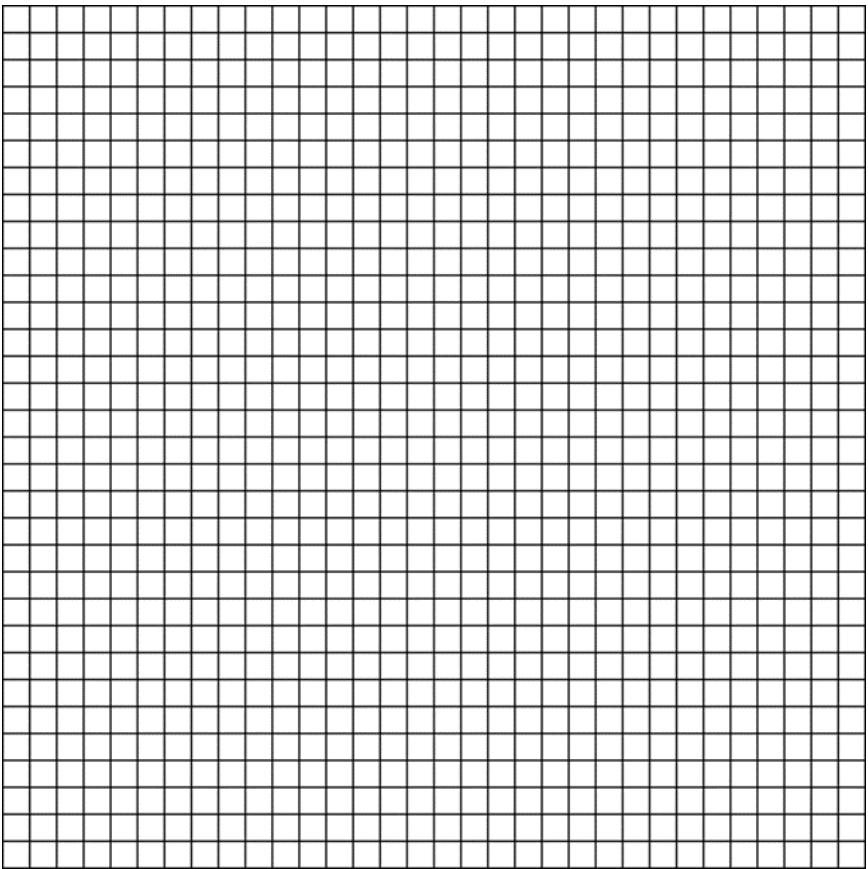
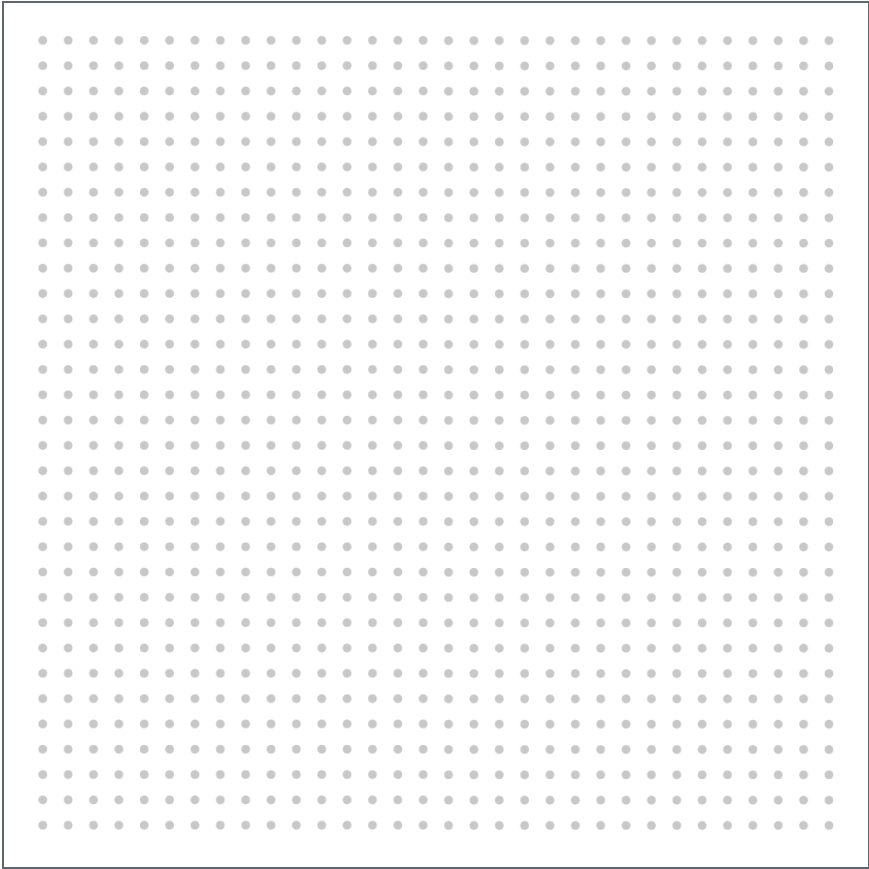


16 x 16 Grid



32 x 32 Grid

Note: With this grid size, the Painter will have an infinite amount of paint. Keep this in mind while you plan your algorithms to create your design.



Step 2: Decompose the Problem

New subclass: SunPainter

Extends: PainterPlus

Method	paintSunCore(String color)
---------------	-----------------------------------

Pseudocode

```
paint the current square with color          // this is the center of the sun
for each direction in {north, south, east, west}:
    turn to face that direction
    if the painter can move forward:
        move forward one square
        paint the current square with color
        turn around and move back to the center square
```

Method	paintRay(int length, String color)
---------------	---

Pseudocode

```
repeat "length" times:
    if the painter has paint left:
        paint the current square with color
        move forward one square

// called once per direction after turning the painter to face
// north, south, east, or west from the tip of the sun
```

End of Project Reflection

Describe your project.

I built a "Neighborhood Sunburst" mural on an 8x8 grid. A SunPainter object paints a small plus-shaped sun in gold at the center of the grid using paintSunCore, then calls a second method, paintRay, four times (turning between calls) to paint orange rays stretching from the sun out to each edge of the grid, so the mural reads like a sun rising over the neighborhood.

What are two things about your project that you are proud of?

- paintRay works for any length just by changing one argument, so the same method draws all four rays instead of me writing four almost-identical methods.
- The sun looks rounder than a plain square block, even though it's only five squares, because I only painted the four squares touching the center instead of filling a solid square.

Describe something you would improve or do differently if you had an opportunity to change something about your project.

I'd add a second subclass, like a CloudPainter, so the mural had more than one kind of shape instead of just the sun and its rays. I'd also center the sun on a bigger grid so all four rays could be the same length instead of two short rays and two long ones.